

BBF-412-1 WEB AUTOMATION TESTING WITH AATF PYTHON

AATF

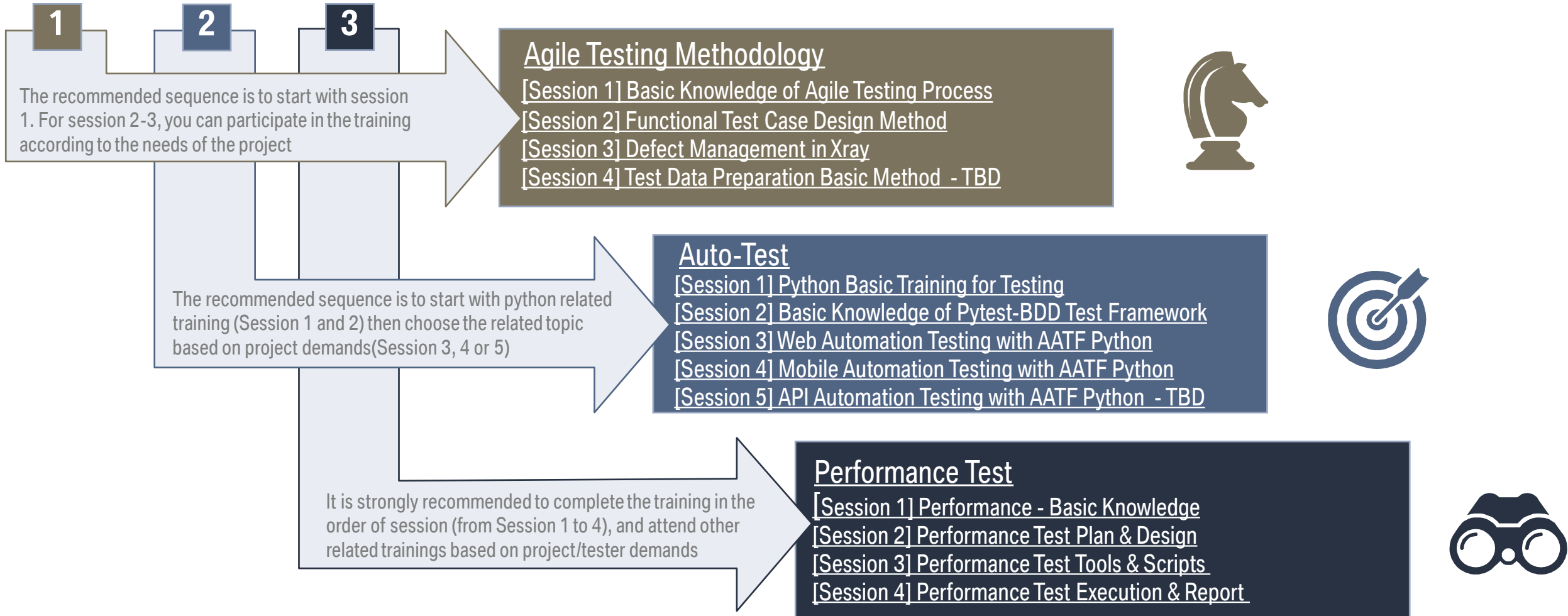
Agile Testing Coach Team

BMW BRILLIANCE
AUTOMOTIVE



BACKGROUND

According to AWM is the Feature Team responsible of the generated quality of the product (including execution of functional / non-functional tests, according to specifications). Below is the Qualification Map Testing, it will help you to find the right training course, points out the mandatory training courses and shows the available training courses per competence area.



AATF (ATC Automation Test Framework)

GUI

(Gherkin Test/DIY Selenium Method)

API

(Rest-assured;Jmeter;Sopui etc.)

Mobile

SAP

How to apply?

(024) 8455 5555 or E-mail Liying.Li@bmw-brilliance.cn

AGENDA

- Automation GUI with AATF Gherkin(Cucumber)
 - ✓ Basic knowledge of Gherkin language: Keywords
 - ✓ Basic knowledge of Web Testing
 - ✓ Set-up Local Test environment
 - ✓ Generate Test Scripts
 - ✓ Test management with Xray

Part 1: Basic knowledge of Gherkin language: Keywords

GHERKIN KEYWORDS IN AATF

Gherkin is not necessarily used to write automated tests. Gherkin is primarily used to write structured tests which can later be used as project documentation.

Primary keyword

- **Feature**

- **Background**

- **Scenario**

- **Given**

- **When**

- **Then**

- **And**

- **But**

- *****

Secondary keyword

- **"""** Multiple lines comments **"""**

- **@** Tag

- **#** Single line comments

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

Each Gherkin file begins with a **Feature** keyword. Feature defines the logical test functionality you will test in this feature file. For e.g if you are testing a payment gateway your Feature will become Payment Gateway or if you are testing the LogIn functionality then the Feature will become Login. The idea of having a feature file is to put down a summary of what you will be testing. **This will serve as the documentation for your tests as well as a good point to start for a new team member.** Note that a feature keyword is present at the starting of the feature file.

Feature : LogIn Action Test

Or

Feature : LogIn Action Test

Description: This feature will test a LogIn and LogOut functionality

This feature will test a LogIn and LogOut functionality

Notice that whatever comes after the **Feature: keyword, will be considered as the feature description.** Feature description can span across multiple lines like shown above in second example. Everything after Feature: till the next Keyword is encountered is considered as feature description.

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

Background Keyword

Background keyword is used to **define steps** which are common to **all the tests** in the feature file.

Feature: Add to Cart

This feature will test functionality of adding different products to the User basket from different flow

Background: User is Logged In

Scenario: Search a product and add the first result/product to the User basket

Given User searched for Lenovo Laptop

When Add the first laptop that appears in the search result to the basket

Then User basket should display with 1 item

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

Scenario: Keyword

Each Feature will contain some number of tests to test the feature. Each test is called a **Scenario** and is described using the Scenario keyword.

Scenario: Search a product and add the first result/product to the User basket

Or

Scenario: Successful Login with Valid Credentials

A scenario is equivalent to a test in our regular development process. Each scenario/test can be basically broken down into three parts:

- Precondition to the test, which represent with (Given) keyword
- Test step execution, which represent with (When) keyword
- Verification of the output with expected result, which represent with (Then)

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

Given Keyword

Given defines a precondition to the test.

For e.g. In shopping website, assume that the LogIn page link is only present on the Home Page, so the precondition for clicking the LogIn link is that the user is at the Home Page. If user is not at the Home Page, user would not be able to enter Username & Password.

This precondition can be expressed in Gherkin like this:

Scenario: Successful LogIn with Valid Credentials

Given User is on Home Page

When User Navigate to LogIn Page

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

When Keyword

When keyword defines the test action that will be executed. By test action we mean the user **input action**.

Scenario: Successful LogIn with Valid Credentials

Given User is on Home Page

When User Navigate to LogIn Page

Here user is performing some action using When keyword, clicking on the LogIn link. We can see that when defines the action taken by the user. It's the event that will cause the actual change in state of the application.

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

Then Keyword

Then keyword defines the **Outcome of previous steps**. We can understand it best by looking at the test above and adding a Then step there.

Feature: LogIn Action Test

Description: This feature will test a LogIn and LogOut functionality

Scenario: Successful Login with Valid Credentials

Given User is on Home Page

When User Navigate to LogIn Page

And User enters UserName and Password

Then Message displayed LogIn Successfully

Here we can see that Then is the outcome of the steps above. **The reader of this test would easily be able to relate to Then step and would understand that when the above conditions are fulfilled then the Then step will be executed.**

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

And Keyword

And keyword is used to add conditions to your steps. Let's look at it by modifying our example a little

Feature: Login Action Test

Description: This feature will test a Login and Logout functionality

Scenario: Successful Login with Valid Credentials

Given User is on Home Page

When User Navigate to Login Page

And User enters UserName and Password

Then Message displayed Login Successfully

Here you would see that And is being used to add more details to the Given step, it's simply adding more conditions. We have just added three conditions. **Use it when you have specified more than one condition. And is used to add more conditions to Given, When and Then statements.**

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

But keyword is used to **add negative type comments**. It is not a hard & fast rule to use but only for negative conditions. It makes sense to use But when you will try to add a condition which is opposite to the premise your test is trying to set. Take a look at the example below:

Feature: LogIn Action Test

Description: This feature will test a LogIn and LogOut functionality

Scenario: Unsuccessful Login with InValid Credentials

Given User is on Home Page

When User Navigate to LogIn Page

And User enters UserName and Password

But The user credentials are wrong

Then Message displayed Wrong UserName & Password

Here you can see how adding But has helped define a negative test, in this test we will try to test failure conditions. Where a wrong credentials are a failure condition.

GHERKIN KEYWORDS IN AATF

– Xray support below keywords:

* Keyword

This keyword is very special. This keyword defies the whole purpose of having Given, When, Then and all the other keywords. Basically Cucumber doesn't care about what Keyword you use to define test steps, all it cares about what code it needs to execute for each step. That code is called a step definition and we will discuss about it in the next section. At this time just remember that all the keywords can be replaced by the * keyword and your test will just work fine. Let's see with example, we had this test earlier:

Feature: LogIn Action Test

Description: This feature will test a LogIn and LogOut functionality

Scenario: Successful Login with Valid Credentials

Given User is on Home Page

When User Navigate to LogIn Page

And User enters UserName and Password

Then Message displayed Login Successfully

Using * Keyword

Feature: LogIn Action Test

Description: This feature will test a LogIn and LogOut functionality

Scenario: Successful Login with Valid Credentials

- * User is on Home Page
- * User Navigate to LogIn Page
- * User enters UserName and Password
- * Message displayed Login Successfully

Part 2: Basic Knowledge of Web Testing

A web page consists of a set of different web elements. We need to know what are the different elements on a web page and then how to locate the web elements.

Here are some common web elements we encounter most of the time while automating the web page.

- Text box: https://www.w3school.com.cn/tiy/t.asp?f=eg_html_input_text
- Drop-down List: https://www.w3school.com.cn/tiy/t.asp?f=eg_html_elements_select
- Checkbox: https://www.w3school.com.cn/tiy/t.asp?f=eg_html_input_checkbox
- Radio button: https://www.w3school.com.cn/tiy/t.asp?f=eg_html_input_radio
- Hyperlink: https://www.w3school.com.cn/tiy/t.asp?f=eg_html_a_href
- TextArea: https://www.w3school.com.cn/tiy/t.asp?f=eg_html_elements_textarea
- Image: https://www.w3school.com.cn/tiy/t.asp?f=eg_html_image_test
- Button: https://www.w3school.com.cn/tiy/t.asp?f=eg_html_button_test

TYPES OF LOCATORS TO IDENTIFY THE WEB ELEMENTS IN AATF

An element locator is simply a method for finding an element on the web page.

We can use the following locators based on priority to identify the elements on the web page in AATF.

Method	Syntax	Description
By ID	e.g.: I enter "XXXXX" into input field " id =searchCarVal"	Locates an element using the ID attribute
By name	e.g.: I clear input field " name =XXXX"	Locates an element using the Name attribute
By class name	e.g.: I click on element " class =ivu-select-item"	Locates an element using the Class attribute
By link text	e.g.: I click on link having text "XXXX"	Locates a link using link text
By partial link text	e.g.: I click on link having partial text "Welcome To SAP"	Locates a link using the link's partial text
By CSS	e.g.: I click on element " css =div[class='nexnav']>ul>:nth-child(4)>button"	Locates an element using the CSS selector
By Xpath	e.g.: I select "lessThan18" option by text from dropdown " xpath =./li[@class='ivu-select-item'][2]"	Locates an element using XPath query
By Index	e.g.: I select " 1 " option by index from multiselect dropdown " xpath =./li[@class='ivu-select-item'][2]"	Locaters an element using Index

USEFUL TOOLS FOR XPATH GENERATION

Useful tools to help with XPath generation

– **SelectorsHub**

SelectorsHub is the free Next Gen XPath plugin & CSS Selectors plugin, helps to generate and verify XPath, cssSelector.

– **XPath Helper**

XPath helper is easy to edit, extract and evaluate queries of XPath.

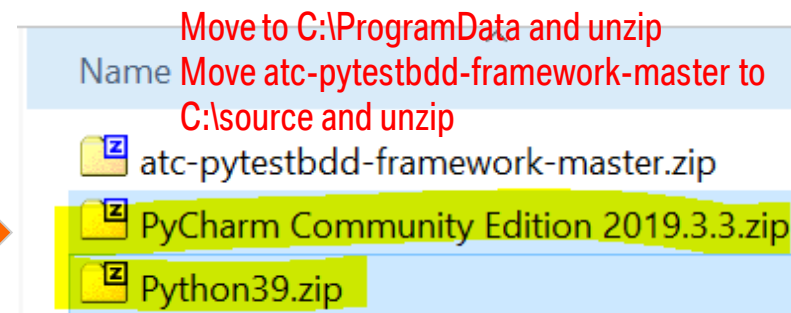
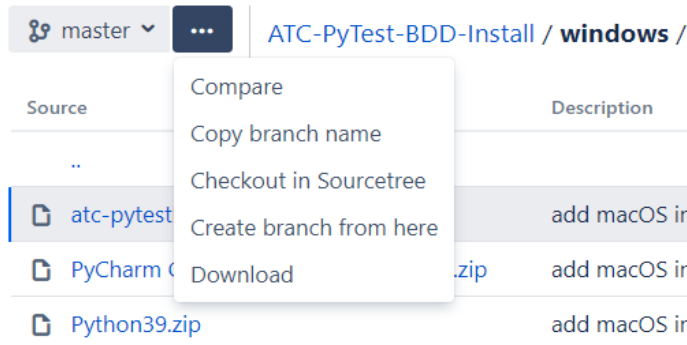
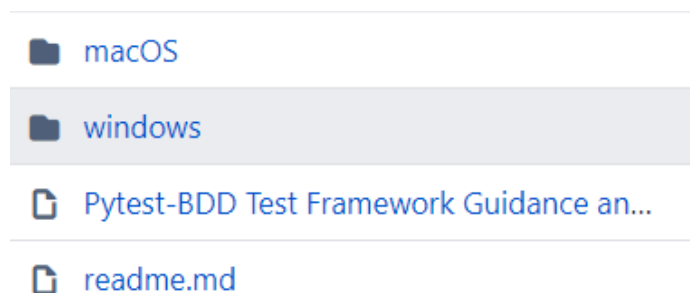
– **AI XPath**

Chrome extension script for getting element's XPath automatically.

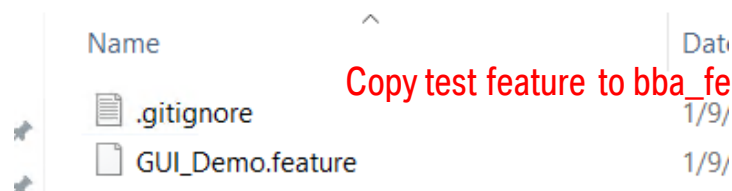
Part 3: Set-up Local Test environment

TEST ENVIRONMENT SETUP – ATC PYTESTBDD INSTALL

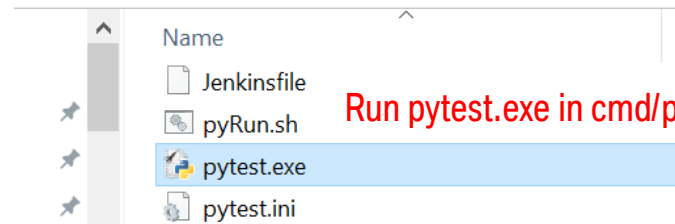
ATC-PyTest-BDD-Install, refer to readme



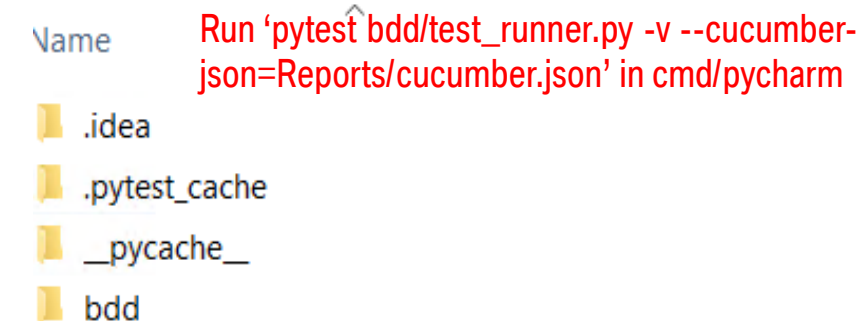
atc-pytestbdd-framework-master > bdd > bba_feature



Windows (C:) > source > atc-pytestbdd-framework-master



atc-pytestbdd-framework-master



TEST ENVIRONMENT SETUP – GENERAL OFFICIALLY INSTALL (ANACONDA)

Install software

(Use Anaconda with Python3.9 or ask IT to help install Python3.9)

- [Anaconda, archive](#)
- [Git](#)
- [PyCharm Community](#)

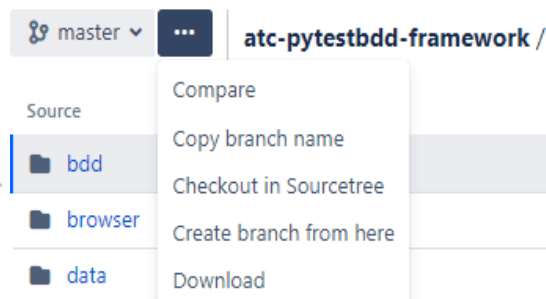


Variable	Value
ANACONDA_HOME	C:\ProgramData\Anaconda3
CLASSPATH	.;C:\Local\Software\Java\jdk1.
JAVA_HOME	C:\Local\Software\Java\jdk1.8
MAVEN_HOME	C:\Local\Software\apache-ma
OneDrive	C:\Users\uf101339\OneDrive
Path	C:\Users\uf101339\AppData\

Environment variable configuration

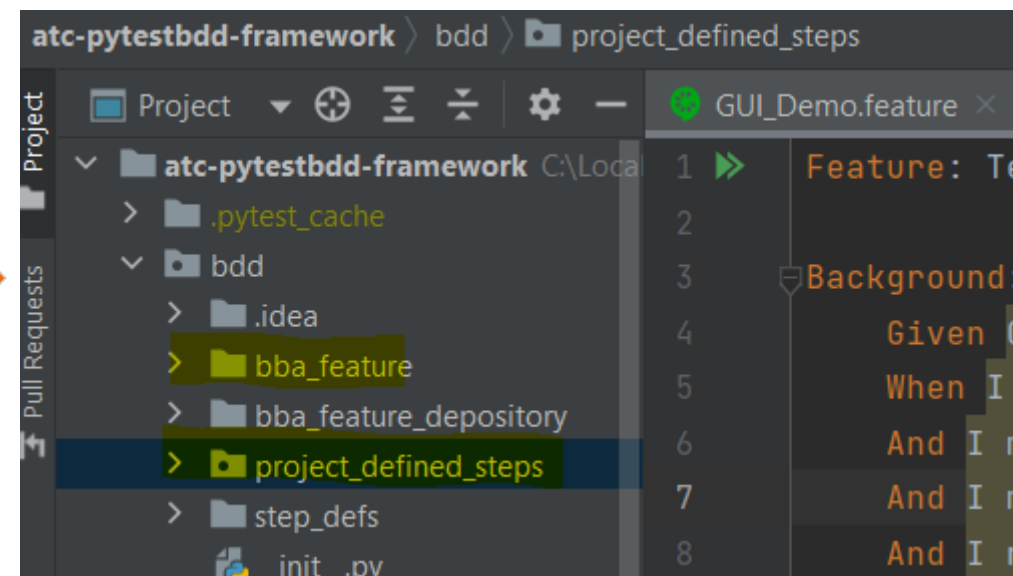
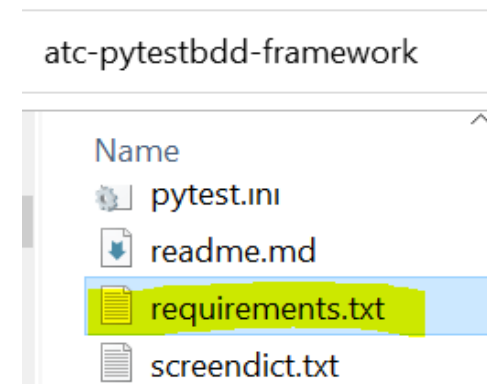
```
%ANACONDA_HOME%
%ANACONDA_HOME%\Lib\site-packages
%ANACONDA_HOME%\Scripts
%ANACONDA_HOME%\Library\bin
%ANACONDA_HOME%\mingw-w64\bin
```

Clone AATF



Install required packages

pip install xxx -i mirror_address
(external bba network)
pip install pytest-bdd-diy==6.1.1.1 -i
<https://atc.bmw-brilliance.cn/nexus/repository/autotest/simple> (internal bba network)



Copy test feature to **bba_feature**, run script command in terminal
Add your own step definition to **project_defined_steps**

Part 4: Generate Test Scripts

PRACTICE 1

<https://demo.seleniumeasy.com/basic-first-form-demo.html>

用例1 – enter \ click

在“Enter a”中输入内容，在“Enter b”中输入内容，点击“Get total”按钮，获取页面中显示的值，与预期结果进行比较。

Sample:

-Open Browser:

Given Open browser and navigate to “XXXXXXX”

-Input value into field:having

I **enter** “XXXX” into input field “**id** =XXXXXX”

I **enter** “XXXX” into element “**id** =XXXXXX”

-Click button:

I **click** on element “**xpath**=XXXXXX”

-Element value check:

element “**xpath** =XXXXXX” should have text “XXXXXX”

element “**xpath** =XXXXXX” should have **attribute** “XXXXXX” with value “XXXXXX”

PRACTICE 2

<https://demo.seleniumeasy.com/basic-checkbox-demo.html>

用例2 – check, uncheck, toggle checkbox

勾选multiple checkbox中的多个选项， 选掉一个已勾选的内容

Sample:

-Check:

I **click** on element “**xpath=XXXXXX**”

-Uncheck:

I **click** on element “XXXXX ”

PRACTICE 3

<https://demo.seleniumeasy.com/basic-radiobutton-demo.html>

用例3 – select radio button group, should have partial text

勾选Sex和Age Group中的内容，点击Get Values，判断页面中显示的值与预期结果进行比较

Sample:

- Radio button group :

I **select** "XXX" **option** by value from **radio button group** "**name =XXX**"

- Should have partial text :

element "XXXX" **should have attribute** "XXXXXX" contains value "XXXXXX"

PRACTICE 4

<https://demo.seleniumeasy.com/jquery-dropdown-search-demo.html>

用例4- by index from multiselect dropdown, should be selected

在列表中搜索字母D，点选其中的两个选项，并判断预期选项已被选中。

Sample:

-by index from multiselect dropdown:

I **select** "XXX" option by **index from dropdown** "**class = XXXX**"

I **select** "XXX" option by **value from dropdown** "**class = XXXX**"

- should be selected:

option "XXXX" by **text** from dropdown "**class =XXXX**" **should be selected**

element "XXXX" **should have partial text** "XXXX"

PRACTICE 5

<https://demo.seleniumeasy.com/bootstrap-date-picker-demo.html>

用例5- current timestamp, create parameter\enter parameter\into parameter\compare parameter

输入当前日期为结束日期，清空开始日期并输入一个未来日，校验结束日期应该自动填充并与开始日期一致

Sample:

-get current timestamp and save to variable:

I save the timestamp with “%d/%m/%Y” as variable “**current_timestamp**”

I enter **\${current_timestamp}** into element “**xpath = XXXXXX**”

- create parameter\enter parameter\into parameter:

I **create parameter** named “XXXX” with value “XXX”

I **enter \${parameter_name}** into input field “**xpath = XXXXX**”

I save element “XXXX” by attribute “XXX” into parameter named “XXX”

I **compare \${parameter_name1}** with **\${parameter_name2}** should have same value

I **compare \${parameter_name1}** with **\${parameter_name2}** should not have same value

PRACTICE 6

<https://demo.seleniumeasy.com/table-search-filter-demo.html>

用例6- each row in the table should have text

在输入框中输入In progress, 判断表格中所有显示的记录中都有in progress

Sample:

-each row in the table should have text:

The element “**xpath =XXXX**” of **each row in the table should have text** including “XXXX”

PRACTICE 7

<https://demo.seleniumeasy.com/javascript-alert-box-demo.html>

用例7- accept alert

在Java Script Confirm Box下，点击click me，在弹出框上是点击OK或Cancel，根据返回结果判断页面显示内容是否正确

Sample:

-accept alert:

| **accept alert**

PRACTICE 8

<https://demo.seleniumeasy.com/basic-first-form-demo.html>

用例8- Scenario outline

使用Scenario outline 将Practice 1重写，实现数据驱动

Sample:

-Scenario outline:

I **enter** "<a>" into input field "XXXXXX"

I **enter** "" into element "XXXXXX"

Example:

lalblcl

l1l2l3l

STATUS @ AUTOMATION

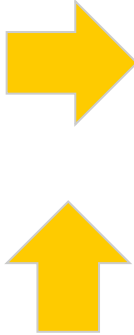
DEMO: AUTOMATION TEST TO CONFIRM THE LOGIN IS SUCCESSFUL AND ADDED BOOK

User Story:

As a user,

I want to login Automation Test Demo Page with the correct username, password and add book

So that I can see the text “ATC Automation Test” is displayed.



Login

ATC Automation Test

☐ GUI Test
 ☐ Auto Test
 ☐ Performance Test
 [Open a test page](#)

Name:
 ID:
 Description:
 Type:
 Price:
 Click on text

Name	ID	Description	Type	Price	Operation	
Frozen	1	Disnep Frozen 冰雪奇缘	Novel	25.8	Delete	Edit
Alice WonderLand	2	Disnep Alice WonderLand 爱丽丝梦游仙境	Novel	28.8	Delete	Edit

```

Scenario: Testcase switch between Windows Functions
When I load element from "Demo.yml"
#Please note that element yml and data yml can not be the same
And I load data from "BookInfor.yml"
And I enter ${GlobalData.LoginUser} into input field ${Demo.username}
And I enter encrypted "QEwd/DWmy/4yGncCqBofQQ==" into input field ${Demo.password}
And I click on element ${Demo.login}
# Read data from yml and parameters
And I enter ${BookInfor.BookName} into input field ${Demo.name}
And I enter ${BookInfor.ID} into input field ${Demo.id}
And I save the timestamp as variable "tmp_time"
And I enter "test_${tmp_time}_Jake" into element ${Demo.Author}
And I take a screenshot
    
```


STATUS @ AUTOMATION

DEMO: AUTOMATION TEST TO CONFIRM THE LOGIN IS SUCCESSFUL AND ADDED BOOK

User Story:

As a user,
I want to login Automation Test Demo Page with the correct username, password and add multiple books.
So that I can see the text “ATC Automation Test” is displayed.

Login

Username

Password

Login

ATC Automation Test

☐ GUI Test
 ☐ Auto Test
 ☐ Performance Test
 [Open a test page](#)

Name:

ID:

Description:

Type:

Price:

Submit

Click on text

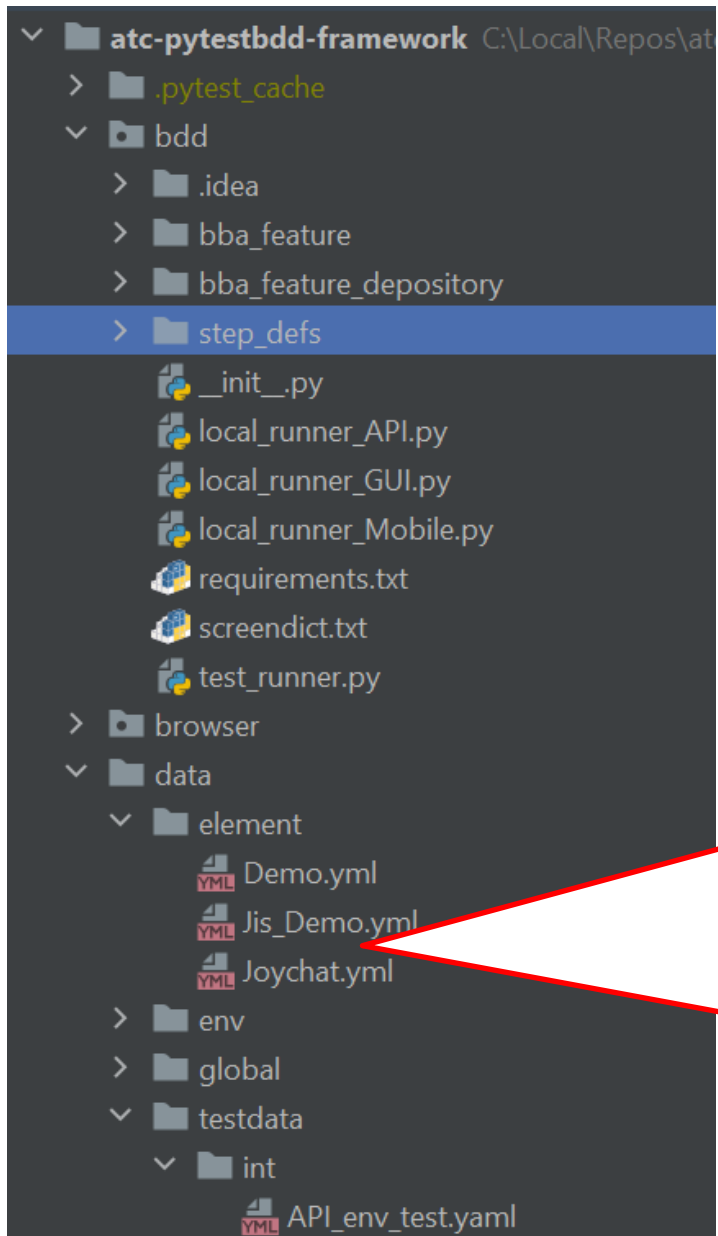
Name	ID	Description	Type	Price	Operation	
Frozen	1	Disnep Frozen 冰雪奇缘	Novel	25.8	Delete	Edit
Alice WonderLand	2	Disnep Alice WonderLand 爱丽丝梦游仙境	Novel	28.8	Delete	Edit

```

Scenario Outline: Testcase switch between Windows Functions
  When I load element from "Demo.yml"
  And I enter ${GlobalData.LoginUser} into input field ${Demo.username}
  And I enter encrypted "QEwd/DWmy/4yGncCqBofQQ==" into input field ${Demo.password}
  And I click on element "id=loginBtn"
  And I enter <name> into input field ${Demo.name}
  And I enter <id> into input field ${Demo.id}
  And I save the timestamp as variable "tmp_time"
  And I enter <author> into element ${Demo.id}
  And I take a screenshot

Examples:
  |name|id|author|
  |woeuy|1|gowiuy|
  |woeuy|1|gowiuy|
  
```

DATA SEPARATION IN PYTEST BDD



```
Scenario Outline: Testcase switch between Windows Functions
  When I load element from "Demo.yml"
  And I enter ${GlobalData.LoginUser} into input field ${Demo.username}
  And I enter encrypted "QEwd/DWmy/4yGncCqBofQQ==" into input field ${Demo.password}
```

```
Demo.yml
1  username:
2    - id=username
3  password:
4    - id=password
5
6  login:
7    - id=loginBtn
8
9  open:
10   - text=Open a test page
11
12  name:
13    - id=bookName
14
15  id:
16    - id = bookQt
17
18  Author:
19    - id = bookAuthor
20
21  end_date:
22    - xpath=//*[@id='datepicker']/input[@placeholder='End date']
```

Part 5: Test management with Xray

RUN AND MAINTAIN AUTOMATION TESTS ON JIRA



AgileTestingCoach / AGILETESTINGCOACH-638

1_input_and_click

[Edit](#)
[Add comment](#)
[Assign](#)
[More](#)
[In Maintenance](#)
[Start Auto Test](#)
[Inactive](#)

Details

Type: **Test** Status: **READY** (View Workflow)
 Priority: **Medium** Resolution: **Unresolved**
 Labels: **CucumberGUI** **Python** **incognito** **int**
 Manual Test Steps: **PRINTABLE FIELD**
 Test Sets association with a Test:
 Prime: **RM2**
 Domain: **DevOps Platform**
 Product: **DevOps Toolchain**

Description

Pre-Conditions

Test Details

Type: **Cucumber**
 Scenario Type: **Scenario**
 Scenario:


```

When Open browser and navigate to "https://demo.seleniumeasy.com/basic-first-form-demo.html"
And I wait for 2 sec
And I load element from "Demo.yml"
And I scroll to end of page
And I wait for 2 sec
And I scroll to top of page
And I wait for 2 sec
And I scroll to element ${Demo.gettotal}
And I enter "20" into input field "id=sum1"
And I enter "50" into element "id=sum2"
And I do JS click on element "xpath=//*[@id='gettotal']/button"
And I wait for 2 sec
And I take a screenshot
Then element "id=displayvalue" should have attribute "innerText" with value "70"
    
```



ATC Testing Xray / ATX-6803

AATF Practice Summary

[Edit](#)
[Add comment](#)
[Assign](#)
[More](#)
[In Maintenance](#)
[Start Auto Test](#)
[Close](#)

Details

Type: **Test Plan** Status: **READY** (View Workflow)
 Priority: **Medium** Resolution: **Unresolved**
 Affects Version/s: **None** Fix Version/s: **None**
 Labels: **None**
 Prime: **RM2**
 Domain: **DevOps Platform**
 Product: **DevOps Toolchain**

Description

[Click to add description](#)

Tests

[Add Tests](#)
[Create Test Execution](#)

Overall Execution Status

7 PASS **1** FAIL

Total Tests: 8

Filter(s)



Show 10 entries

	Key	Summary	Requirements	#Test Executions	Issue Assignee	Dataset
☐	ATX-6794	Practice 1		1	Unassigned	
☐	ATX-6804	Practice 2		1	Unassigned	
☐	ATX-6805	Practice 3		1	Unassigned	
☐	ATX-6806	Practice 4		1	Unassigned	

DEMO: AUTOMATION TEST PLAN RUN ON SCHEDULED

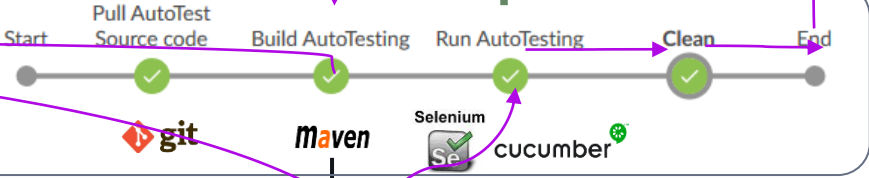


Any Branch

Tigger if finished



Auto Testing Pipeline



URL

Download Dependencies & Upload the artifacts



Q&A

THANK YOU

APPENDIX (1)- XPATH

Xpath是一门在XML文档中查找信息的语言。

在Xpath中，有七种类型的节点：元素、属性、文本、命名空间、处理指令、注释以及文档（根）节点。XML文档是被作为节点树来对待的。树的根被称为文档节点或根节点。

节点关系：父（Parent） 子（children） 同胞（Sibling） 先辈（Ancestor） 后代（Descendant）

参考地址：<http://www.w3school.com.cn/xpath/index.asp>
<https://www.cnblogs.com/songshu120/p/5182043.html>

APPENDIX (1)- XPATH 语法

Xpath使用路径表达式来选取XML文档中的节点或节点集。节点是通过沿着路径（path）或者步（steps）来选取的。

路径表达式	结果	样例
/	路径起始于正斜杠(/), 则此路径代表元素的绝对路径	/html/head
//	提取页面中所有的指定元素不管路径,可嵌套使用	//h1 //div[@id='projects-index']//li
@	选取属性	//@href
*	匹配任何元素节点	//html/*
@*	匹配任何属性节点	//a[@*]
	选取若干路径	//h1 //h2

APPENDIX (1)- XPATH 谓语句

谓语句用来查找某个特定的节点或包含某个指定的值的节点。谓语句被嵌在方括号中

路径表达式	结果	样例
//div[1]	选取第一个div元素	//li[@class='root'][1]
//div[last()]	选取最后一个div元素	//li[@class='root'][last()]
//div[last()-1]	选取倒数第二个div元素	//li[@class='root'][last()-1]
//div[position()<3]	选取前两个div元素	//li[@class='root'][position()<3]
//div[@id='**']	选取指定id的div元素	//div[@id='projects-index']
//div[node>value]	选择值大于指定值的元素	//table[@class='list issue-report']//td[a>600]
//div[contains(node, value)]	选择值包含指定值的元素	//a[contains(@href,'audit')]")

APPENDIX (1)- XPATH 轴

轴可定义相对于当前节点的节点集

轴名称	结果	样例
ancestor	当前节点的所有先辈（父、祖父）	//li[@class='root']/ancestor::div
ancestor-or-self	当前节点本身及先辈节点	//li[@class='root']/ancestor-or-self::*
descendant	当前节点的所有后代（子、孙）	//li[@class='root'][1]/ul/descendant::div
descendant-or-self	当前节点本身及所有后代	//li[@class='root'][1]/descendant::*
child	当前节点的所有子元素	//li[@class='root'][1]/child::ul
parent	当前的节点的父节点	//li[@class='root']/parent::*
self	当前节点	//li[@class='root'][1]/ul/self::*

APPENDIX (1)- XPATH 轴

轴可定义相对于当前节点的节点集

轴名称	结果	样例
ancestor	当前节点的所有先辈（父、祖父）	//li[@class='root']/ancestor::div
ancestor-or-self	当前节点本身及先辈节点	//li[@class='root']/ancestor-or-self::*
descendant	当前节点的所有后代（子、孙）	//li[@class='root'][1]/ul/descendant::div
descendant-or-self	当前节点本身及所有后代	//li[@class='root'][1]/descendant::*
child	当前节点的所有子元素	//li[@class='root'][1]/child::ul
parent	当前的节点的父节点	//li[@class='root']/parent::*
self	当前节点	//li[@class='root'][1]/ul/self::*

轴名称	结果	样例
attribute	当前节点的所有属性	//li[@class='root'][1]/attribute::*
following	选取当前节点之后的所有节点	//li[@class='root'][1]/ul/following::ul
preceding	选取当前节点之前的所有节点	//li[@class='root'][2]/preceding::ul
Preceding-sibling	选取当前节点之前的同级节点	//li[@class='root'][2]/preceding-sibling::*

APPENDIX (2)- CSS

css选择器有44种. 其中常用的有如下23种:

- 1) #element element: div p, 选择div标签元素内部的所有p标签元素
- 2) #element>element: div>p, 选择父元素为div标签元素的所有p标签元素
- 3) #element element: div p, 选择div标签元素内部的所有p标签元素
- 4) #element>element: div>p, 选择父元素为div标签元素的所有p标签元素
- 5) #element+element: div+p, 选择紧接在div元素之后的所有p元素
- 6) #id: #firstname, 选择id='fistname'的所有元素
- 7) #[attritube=value]: [target=_blank], 选择target="_blank"的所有元素
- 8) #[attribute]: [target], 选择带有target属性的所有元素
- 9) #[attritube~=value]: [title~=flower], 选择title属性包含单词'flower'的所有元素
- 10) #:first-child p:first-child, 选择属于父元素的第一个子元素的每个p标签元素
- 11) #[attribute^=value] a[src^='https'], 选择其src属性以'https'开头的每个a标签元素
- 12) #[attribute\$=value] a[src\$='.pdf'], 选择其src属性以'.pdf'结尾的所有a标签元素
- 13) #[attribute*=value] a[src*='abc'], 选择其src属性包含'abc'子串的每个a标签元素
- 14) #:first-of-type p:first-of-type, 选择父元素的首个p标签元素的每个p标签元素 (所有p标签对应的父标签的第一个子p标签元素)

APPENDIX (2)- CSS

- 15) `#:last-of-type p:last-of-type`, 选择父元素的最后一个p标签元素的每个p标签元素 (所有p标签对应的父标签的最后一个子p标签元素)
- 16) `#:only-of-type p:only-of-type`, 选择父元素里面唯一的p标签元素的每个p标签元素 (所有p标签对应的父标签元素只有一个子p标签元素)
- 17) `#:nth-child(n) p:nth-child(2)`, 选择属于父元素的第二个子元素的每个p标签元素 (所有的p标签的父标签对应的第二个子标签元素)
- 18) `#:only-child p:only-child`, 选择父元素里面只有一个标签的每个p标签元素 (父标签里面只有一个p标签)
- 19) `#:nth-last-child(n) p:nth-last-child(2)`, 选择属于父元素的倒数第二个子元素的每个p标签元素 (所有的p标签的父标签对应的倒数第二个子标签元素)
- 20) `#:nth-of-type(n) p:nth-of-type(2)`, 选择属于其父标签元素的第二个p标签的每个p标签元素 (所有p标签对应的父标签的第二个p标签元素, 也就是这个和p标签同级)
- 21) #鼠标放上悬浮选择菜单
`ActionChains(browser).move_to_element(article).perform()`需要导入
`ActionChains`包
`from selenium.webdriver.common.action_chains import ActionChains`
- 22) `#:nth-last-of-type(n) p:nth-last-of-type(2)`, 选择属于其父标签元素的倒数第二个p标签的每个p标签元素 (所有p标签对应的父标签的倒数第二个p标签元素, 也就是这个和p标签同级)
- 23) `#:last-child p:last-child`, 选择属于其父标签元素最后一个子元素是p标签的每个p标签元素 (父标签的最后一个标签必须是p标签)